

Local LLM Context Access Experiments 발표 대본 - 쉬운 설명 버전

예상 발표 시간: 약 10분

발표 팁: 대괄호 안 문장은 화면 조작 안내입니다. 발표할 때는 읽지 말고 행동 지시로 사용하세요.

핵심 원칙: 영어 용어는 슬라이드에 적힌 이름을 확인할 때만 쓰고, 설명은 최대한 쉬운 한국어로 풀어 말합니다.

1. Overview - 0:00-0:45

안녕하세요. 오늘 발표할 주제는 로컬 LLM 코드 어시스턴트가 큰 자료를 어떻게 다룰 수 있는지에 대한 실험입니다.

LLM은 한 번에 읽을 수 있는 글의 양이 정해져 있습니다. 이 한계를 흔히 context window라고 부르는데, "LLM의 책상 크기"로 보겠습니다.

이 책상 위에 종이를 너무 많이 올리면 두 가지 문제가 생깁니다. 첫째, 서류의 양이 너무 많다면 아예 다 못 올립니다. 둘째, 올리더라도 필요한 한 줄을 찾는 데 오래 걸리고, 비슷하지만 틀린 값을 보고 헛갈릴 수 있습니다.

한학기 동안 진행된 실험의 결론은, 큰 자료를 LLM에게 통째로 주지 말고, 큰 자료는 밖에 보관한 뒤 필요한 부분만 작게 꺼내서 주는 것이 더 좋았다는 것입니다.

2. Target Result - 0:45-1:25

[Target Result 슬라이드를 보여줍니다.]

이번 실험의 목표는 "필요한 증거만 골라서 보여주는 코드 어시스턴트"입니다.

예를 들어 긴 로그 파일이 있다고 하겠습니다. 이 로그 전체를 LLM에게 주면 책상 위가 꽉 찹니다. 대신 로그 파일은 원래 위치에 그대로 두고, 그 파일에 번호표를 붙입니다. 그러면 LLM은 번호표와 짧은 설명만 봅니다. 필요한 경우에는 전용 도구가 그 번호표를 보고 실제 파일을 열고, 필요한 줄만 꺼내 줍니다.

이 번호표를 슬라이드에서는 pointer라고 부릅니다. 오늘은 pointer를 "번호표"라고 생각하시면 됩니다. 큰 상자 자체를 들고 다니는 것이 아니라, 상자 번호만 들고 다니다가 필요할 때 상자를 열어 필요한 물건만 꺼내는 방식입니다.

3. 왜 전체를 넣으면 안 되는가 - 1:25-2:10

[Local LLM Limits 슬라이드의 Raw Prompt Token Pressure 차트를 가리킵니다.]

차트를 보면 전체 자료를 그대로 넣는 방식이 얼마나 큰지 알 수 있습니다. 실행 중 생긴 결과 전체를 넣으면 약 13만 token, 디스크 파일 전체를 넣으면 약 12만 token, 코드 저장소 전체를 넣으면 약 6만 7천 token이 필요했습니다.

token은 LLM이 글을 잘게 나눠 읽는 조각입니다. 숫자가 클수록 읽을 양이 많다는 뜻입니다. 이번 실험의 주된 설정은 약 3만 2천 token 정도를 받을 수 있었기 때문에, 12만 token이나 13만 token은 너무 큼니다.

또 하나 중요한 점은, 많이 넣는다고 더 똑똑해지는 것이 아니라는 점입니다. 오히려 관련 없는 정보가 많아지면 모델이 정답과 비슷한 오답을 고를 수 있습니다. 그래서 이번 실험은 "많이 넣기"가 아니라 "필요한 것만 찾기"를 목표로 했습니다.

4. 네 가지 방법 한눈에 보기 - 2:10-2:55

[Solution Overview 슬라이드를 보여줍니다.]

이번에는 네 가지 방법을 실험했습니다.

첫 번째는 코드 번호표 방식입니다. 슬라이드에는 `symbol_index`라고 적혀 있습니다. 코드 안의 함수, 클래스, 상수 같은 이름 붙은 조각에 번호표를 붙이고, 필요한 코드 조각만 읽는 방식입니다.

두 번째는 책상의 크기를 키우는 방식입니다. 슬라이드에는 `self_extend`라고 적혀 있습니다. LLM이 더 긴 글을 받아 들일 수 있게 하는 방식입니다. 다만 책상이 커진다고 해서 필요한 물건을 잘 찾는지는 따로 확인해야 합니다.

세 번째는 실행 결과 번호표 방식입니다. 슬라이드에는 `runtime_pointer`라고 적혀 있습니다. 테스트 로그, 빌드 결과, API 응답처럼 실행 중 새로 생긴 큰 결과물에 번호표를 붙이는 방식입니다.

네 번째는 외부 파일 번호표 방식입니다. 슬라이드에는 `source_pointer`라고 적혀 있습니다. 이미 디스크에 있는 로그, 문서, JSON 파일에 번호표를 붙이는 방식입니다.

즉 네 방법은 모두 "LLM 책상 위에 무엇을 얼마나 올릴 것인가"를 다르게 해결하려는 시도입니다.

5. Code Number Visualizer - 2:55-4:05

[Symbol Bundle Visualizer를 엽니다. 왼쪽 Source Files, 가운데 목록, 오른쪽 코드 꾸러미 영역을 차례로 가리킵니다.]

먼저 코드 번호표 방식입니다. 슬라이드에는 `symbol_index`라고 나오지만, 쉽게 말하면 "코드 주소록"입니다.

[step 1] 코드에는 이름 붙은 조각들이 있습니다. 함수, 클래스, 메서드, 상수 같은 것들입니다. 이런 코드 조각을 슬라이드에서는 symbol이라고 부릅니다. 오늘은 그냥 "이름 붙은 코드 조각"이라고 이해하시면 됩니다.

[step 2] 첫 단계에서는 로컬 프로그램이 소스 파일을 읽고, 이름 붙은 코드 조각을 찾아냅니다. 이때 LLM이 전체 코드를 읽는 것이 아닙니다. 프로그램이 미리 주소록을 만드는 것입니다.

[가운데 Symbol Index 영역을 가리킵니다.]

[step 3] 주소록에는 코드 본문 전체가 들어가지 않습니다. 대신 "어느 파일의 몇 번째 줄부터 몇 번째 줄까지 어떤 함수가 있다" 같은 정보가 들어갑니다.

[Candidate Outline 영역을 가리킵니다.] [step 4] 화면의 Candidate Outline은 그냥 "후보 목록"입니다. 사용자가 질문하면 질문 속 단어와 관련 있어 보이는 함수나 클래스를 먼저 추립니다. 예를 들어 "만료된 토큰에서 어떤 에러가 나오요?"라고 물으면 `validate_token`, `expired`, `TOKEN_EXPIRED_MESSAGE` 같은 단어를 단서로 삼아 후보를 줄입니다.

[Selected URI 영역을 가리킵니다.] [step 5] 그 다음 LLM은 이 작은 후보 목록만 보고 어떤 코드 조각을 열어볼지 고릅니다. 중요한 점은 여기서 LLM이 아직 코드를 읽고 답하는 것이 아니라는 점입니다. 이 단계는 "어느 서랍을 열지 고르는 단계"입니다.

[Resolve and Bundle 영역을 가리킵니다.] [step 6] 오른쪽의 Resolve and Bundle도 쉽게 말하면 두 단계입니다. 첫째, 번호표를 실제 파일 위치와 줄 범위로 바꿉니다. 둘째, 그 함수만 달랑 꺼내는 것이 아니라, 답에 필요한 주변 코드도 함께 묶습니다.

예를 들어 `build_cache_key` 함수 안에서 `sanitize_report_name`이라는 함수를 부른다면, `build_cache_key`만 읽어서는 최종 결과를 정확히 알기 어렵습니다. 그래서 관련된 함수와 상수까지 작은 꾸러미로 묶습니다. 이를 bundle이라고 부릅니다.

이 방식은 전체 코드를 통째로 읽지 않고, 코드 주소록을 만들고, 후보를 줄이고, 번호표 하나를 고르고, 필요한 주변 코드까지 작은 묶음으로 만들어 LLM에게 준다는 것입니다.

6. Self-Extend Visualizer - 4:05-5:00

[Self-Extend Visualizer를 엽니다. Base Position Grid, Overflow, Grouped Position Layer, Local Neighbor Window를 순서대로 가리킵니다.]

[step 1] 이 방식은 앞에서 본 번호표 방식과 다릅니다. 직전의 솔루션은 큰 자료를 밖에 두고 필요한 부분만 가져오는 방식입니다. Self-Extend는 LLM이 더 긴 글을 받을 수 있게 책상 자체를 넓혀보는 방식입니다.

visualizer에서 8개의 칸은 기본 책상 크기, 즉 context window를 단순화해서 보여줍니다. 짧은 글은 각 token이 자기 칸을 하나씩 차지합니다.

[step 2] [Overflow 단계로 이동합니다.]

글이 길어지면 빨간색 영역이 생깁니다. 이것은 책상 밖으로 넘친 글입니다. 기본 상태에서는 더 넣을 자리가 없습니다.

[step 3] [Grouped Position Layer 단계로 이동합니다.]

[step 4-5] Self-Extend를 켜면 오래된 token들을 더 거칠게 묶어서 넣습니다. 쉽게 말하면, 멀리 있는 내용은 흐릿하게 압축해서 보고, 최근 내용은 선명하게 보는 방식입니다. 보라색 영역은 멀리 있는 내용을 압축해서 보는 부분이고, 초록색 영역은 최근 내용을 비교적 선명하게 보는 부분입니다.

[step 6] 여기서 꼭 구분해야 합니다. "긴 글을 받을 수 있다"와 "긴 글 속 정답을 정확히 찾을 수 있다"는 다릅니다. visualizer 마지막의 KEY token처럼 정답이 글 안에 들어 있어도, 너무 멀리 있고 흐릿하게 압축된 상태라면 모델이 못 찾을 수 있습니다.

7. Runtime Pointer Visualizer - 5:00-5:55

[Pointer Data Visualizer를 열고 Runtime track을 선택합니다.] [step 1] 이제 실행 결과 번호표 방식입니다. 슬라이드에는 `runtime_pointer`라고 적혀 있습니다.

[step 2] 예를 들어 테스트를 실행했더니 아주 긴 실패 로그가 나왔다고 하겠습니다. 이 로그는 방금 실행 중에 새로 생긴 자료입니다. 이런 자료를 실험에서는 RuntimeStore라는 보관함에 저장했습니다. RuntimeStore는 "runtime에 생성된 데이터들을 잠시 넣어 두는 보관함"입니다.

보관함에 로그를 넣으면 `runtime://log/collect_checkout_trace/0001` 같은 번호표가 생깁니다. LLM에게는 긴 로그 전체가 아니라 이 번호표, 자료 종류, 사용할 수 있는 도구, 짧은 힌트만 보여줍니다.

[Pointer Catalog 영역을 가리킵니다.] [step 3] 화면 가운데의 목록은 "번호표 목록"입니다. 여기에는 실제 긴 로그 내용이 없습니다. 대신 "이 번호표는 로그이고, timeout이라는 단어와 관련 있어 보이며, grep_runtime_log라는 도구로 필요한 줄을 찾을 수 있다" 정도의 정보만 있습니다.

[Tool Selected, Pointer Resolved, Evidence Extracted 순서로 가리킵니다.]

사용자가 "checkout이 왜 실패했나요?"라고 물으면, LLM은 어떤 도구를 쓸지 고릅니다. 예를 들어 timeout이라는 단어 주변 줄을 찾아 달라고 할 수 있습니다.

[step 4 - 6] 그러면 llm이 아닌, LLM이 호출한 도구가 번호표를 보고 보관함에서 실제 로그를 찾고, 그중 필요한 줄 2~3개만 꺼냅니다.

[step 7] 마지막으로 LLM 책상 위에는 도구가 전달한 질문, 번호표, 도구 사용 기록, 그리고 추출된 작은 증거만 올라갑니다. 긴 로그 전체는 끝까지 LLM에게 들어가지 않습니다.

8. Source Pointer Visualizer - 5:55-6:40

[같은 Pointer Data Visualizer에서 Source track을 선택합니다.]

[step 1] 외부 파일 번호표 방식도 거의 비슷합니다. 차이는 가리키는 대상입니다.

이 솔루션은 runtime에 생성된 데이터가 아닌, 이미 디스크에 있던 파일을 포인터를 통해 가리킵니다. 예를 들어 운영 로그 파일, 마크다운 문서, JSON 설정 파일 같은 것입니다.

[step 2] 이 manifest는 "번호표 장부"입니다. 어떤 번호표가 어떤 파일을 가리키는지, 파일 종류가 로그인지 문서인지 JSON인지, 어떤 도구로 열 수 있는지를 적어 둔 장부입니다.

예를 들어 `source://log/logs/checkout_gateway.log`라는 포인터가 있으면, 포인터가 이것을 실제 파일 위치와 연결해 줍니다. 그리고 로그라면 필요한 단어 주변 줄만 찾고, 문서라면 관련 section만 찾고, JSON이라면 필요한 값 하나만 꺼냅니다.

정리하면 이렇습니다. 실행 결과 번호표는 "방금 만들어진 큰 결과물"에 붙이는 번호표이고, 외부 파일 번호표는 "원래 디스크에 있던 큰 파일"에 붙이는 번호표입니다. 둘 다 큰 자료 전체를 LLM에게 주지 않기 위한 방법입니다.

9. Symbol Results - 6:40-7:35

[Symbol Results 슬라이드를 보여줍니다.]

결과를 보기 전에, 차트에 나오는 비교 방식을 먼저 쉽게 설명하겠습니다.

`full_repo`는 코드 저장소 전체를 LLM에게 주는 방식입니다. 가장 단순하지만 가장 무겁습니다.

`full_file`은 정답이 들어 있는 파일 하나를 통째로 주는 방식입니다. 전체 저장소보다는 작지만, 이미 정답 파일을 알고 있다는 가정이 필요합니다.

`line_span`은 정답이 있는 줄 범위만 주는 방식입니다. 예를 들어 20번째 줄부터 25번째 줄까지만 주는 것입니다.

`symbol_oracle`은 사람이 정답 함수가 어디 있는지 이미 안다고 가정하고, 그 함수 본문만 주는 방식입니다. 최대 성능을 알아보기 위해 만든 방식으로, oracle은 "정답 위치를 미리 아는 이상적인 조건"이라는 뜻입니다.

`symbol_select`는 LLM에게 코드 주소록을 주고, LLM이 필요한 함수 주소를 고르게 하는 방식입니다.

`filtered_select_bundle`은 코드 주소록 전체에서 후보를 먼저 작게 줄인 다음, LLM이 고른 함수와 주변 도우미 함수, 상수까지 함께 묶어서 주는 방식입니다.

[Easy Task Wall Time과 Easy Task Prompt Tokens 차트를 가리킵니다.]

쉬운 문제에서는 필요한 코드만 읽는 방식이 훨씬 빨랐습니다. medium 기준으로 코드 저장소 전체를 넣는 `full_repo`는 약 67,188 token과 약 613초가 필요했습니다. 반면 필요한 줄이나 함수만 읽는 방식은 약 199 token, 약 2초 정도였습니다.

[Selector Cost Improvement 차트를 가리킵니다.]

또 기존 `symbol_select`는 전체 코드 주소록을 LLM에게 많이 보여줘야 해서 약 33,922 token이 필요했습니다. 반면 후보를 먼저 줄이고 필요한 코드 묶음만 만드는 방식은 약 587 token으로 줄었습니다.

[Hard Task Accuracy 차트를 가리킵니다.]

어려운 문제에서는 함수 하나만 읽으면 실패했습니다. 왜냐하면 정답이 함수 안에만 있지 않고, 주변 상수나 헬퍼함수에 걸쳐 있었기 때문입니다. 코드 묶음 방식은 필요한 코드를 더 잘 찾아왔지만, 최종 정확도는 1/3이었습니다.

여기서 중요한 해석은 "필요한 코드를 못 찾아서 실패했다"가 아닙니다. 필요한 코드는 들어왔지만, LLM이 문자열 변환이나 숫자 계산을 틀렸습니다. 그래서 앞으로는 계산이 필요한 질문에서는 LLM에게 머리로 계산시키지 말고, 실제 Python 실행이나 작은 계산 도구로 확인하는 보조 단계가 필요합니다.

10. Self-Extend Results - 7:35-8:20

[Self-Extend Results 슬라이드를 보여줍니다.]

Self-Extend 실험에서는 세 가지 테스트를 했습니다. 먼저 이 세 테스트가 무엇인지 설명하겠습니다.

첫 번째는 Passkey 테스트입니다. 긴 의미 없는 글 중간에 숫자 **16808**을 숨긴 뒤, 마지막에 "아까 숨긴 숫자가 뭐였지?"라고 물어보는 테스트입니다. 긴 글 속에서 특정 정보를 다시 찾아낼 수 있는지 보는 실험입니다.

두 번째는 Code Assistance 테스트입니다. 긴 코드 덩어리 속에서 함수 결과, 어떤 함수가 어떤 함수를 부르는지, 테스트가 왜 실패하는지를 찾게 했습니다. 실제 코드 어시스턴트 상황에 더 가까운 테스트입니다.

[Reported Context Capacity 차트를 가리킵니다.]

결과를 보면 Self-Extend는 책상 크기를 키우는 데는 성공했습니다. 기본 상태는 약 4,320 정도였고, G=2는 약 8,416, G=4는 약 16,608로 늘었습니다.

[Passkey Success Rate와 Code Assistance Success 차트를 가리킵니다.]

하지만 정답 찾기는 좋아지지 않았습니다. Passkey 테스트에서 기본 상태는 일부 짧은 입력을 맞췄지만, G=2와 G=4는 0%였습니다. 코드 어시스턴트 테스트도 전부 0%였습니다.

쉽게 말하면 책상은 커졌지만, 숨겨 둔 쪽지를 잘 찾지는 못했습니다. 그래서 이 방식은 기본 방법으로 쓰기보다는, 정말 긴 배경 정보가 필요할 때 제한적으로 다시 확인하면서 쓰는 후보 정도로 보는 것이 맞습니다.

11. Runtime and Source Pointer Results - 8:20-9:15

[Runtime Pointer Results 슬라이드를 보여줍니다.]

실험의 비교 방식 먼저 설명하겠습니다.

full_runtime은 실행 중 생긴 큰 결과물을 통째로 LLM에게 주는 방식입니다. 예를 들어 긴 테스트 로그 전체를 그대로 넣는 것입니다.

pointer_oracle은 사람이 이미 어디를 봐야 하는지 안다고 가정하고, 필요한 부분만 꺼내서 LLM에게 주는 방식입니다.

pointer_select는 LLM이 번호표 목록과 도구 설명을 보고, 어떤 도구로 어떤 부분을 꺼낼지 직접 고르는 방식입니다.

결과는 의도한대로 나왔습니다. 전체 실행 결과를 통째로 넣은 방식은 3문제 중 1문제만 맞혔습니다. 반면 포인터 방식 두 개는 3문제를 모두 맞혔습니다. 읽을 양도 131,136 token에서 757 token 또는 311 token으로 크게 줄었습니다.

[Source Pointer Results 슬라이드로 이동합니다.]

외부 파일 포인터 실험도 비교 방식은 같습니다.

full_source는 디스크에 있는 큰 파일 전체를 LLM에게 주는 방식입니다. 예를 들어 운영 로그 전체나 JSON 파일 전체를 넣는 것입니다.

pointer_oracle은 사람이 이미 필요한 위치를 안다고 가정하고, 필요한 부분만 꺼내서 주는 방식입니다.

pointer_select는 LLM이 파일 번호표 목록을 보고, 로그 검색 도구를 쓸지, 문서 section 추출 도구를 쓸지, JSON 값 추출 도구를 쓸지 고르는 방식입니다.

medium 기준으로 전체 파일을 넣은 방식은 3문제 중 1문제만 맞혔습니다. 번호표 방식은 3문제를 모두 맞혔습니다. 읽을 양도 120,208 token에서 약 780 token 또는 286 token으로 줄었습니다.

두 실험의 공통 결론은 이것입니다. 자료가 커져도 번호표 방식은 LLM에게 보여주는 양이 거의 늘지 않았습니다. 큰 자료와 LLM 책상을 분리했기 때문입니다.

12. Overall Conclusion and Next Steps - 9:15-10:00

[Overall Conclusion 슬라이드를 보여줍니다.]

전체 결론을 정리하겠습니다.

코드를 볼 때는 전체 코드를 다 넣지 말고, 필요한 함수와 그 주변 코드만 작은 묶음으로 만들어 주는 것이 좋습니다.

이미 디스크에 있는 큰 로그, 문서, JSON은 파일 전체를 넣지 말고, 번호표와 도구를 이용해 필요한 부분만 꺼내는 것이 좋습니다.

테스트 실행 결과처럼 방금 생긴 큰 출력도 그대로 넣지 말고, 보관함에 넣고 번호표만 전달하는 것이 좋습니다.

LLM의 책상을 키우는 방법도 실험했지만, 이번 결과에서는 책상이 커졌다고 해서 정답을 잘 찾지는 못했습니다. 그래서 핵심은 "더 많이 넣기"가 아니라 "필요한 것을 더 정확히 골라 넣기"입니다.

[Next Steps 슬라이드로 이동합니다.]

앞으로는 이 세 가지를 합치면 됩니다. 테스트 로그는 runtime pointer로 관리하고, 운영 문서나 JSON은 disk pointer로 관리하고, 수정해야 할 코드는 코드 주소록과 코드 묶음으로 찾습니다. 그리고 숫자 계산이나 문자열 계산은 LLM에게 추측하게 하지 말고, 실제 실행 도구로 확인합니다.

최종적으로 만들고 싶은 코드 어시스턴트는 모든 자료의 전체를 읽는 어시스턴트가 아닙니다. 로컬 llm의 한계를 극복하기 위해 필요한 증거를 찾아서 작게 보여주고, 그 근거로 답하거나 코드를 고치는 어시스턴트입니다.

이상입니다. 감사합니다.